Relatório II - Prática: Python: Criando um ReAct Agent do Zero

Danilo Sucupira Galvão

Descrição da atividade

Conceito de Agente e Ciclo ReAct: Um agente é definido como um loop automatizado que interage com um modelo de linguagem (LLM). O padrão ReAct funciona através de um ciclo iterativo de três etapas: **Thought** (Pensamento), **Action** (Ação) e **Observation** (Observação).

Uso da API Groq e Seleção de Modelos: O código detalha a infraestrutura de conexão, utilizando a biblioteca `groq` para instanciar um cliente e realizar chamadas ao modelo *llama-3.3-70b-versatile*. Isso exemplifica como configurar chaves de API e parâmetros de inferência para obter respostas otimizadas para agentes.

O Papel do System Prompt: O prompt do sistema é a "constituição" do agente, instruindo-o a descrever seus pensamentos, escolher uma ação das ferramentas disponíveis e, crucialmente, retornar a palavra **PAUSE**. Isso sinaliza ao código Python que o modelo terminou o raciocínio e está aguardando o resultado de uma ferramenta.

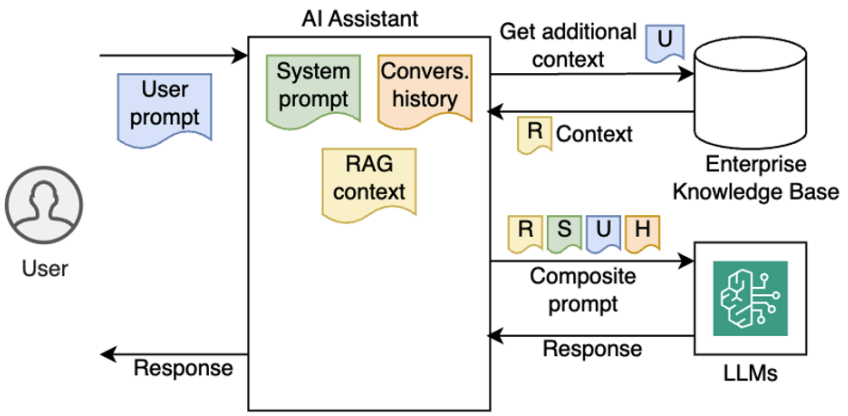
Criação da Classe Agent: É demonstrada a construção de uma classe em Python que gerencia o cliente da API (Groq), mantém o histórico de mensagens e executa as chamadas ao modelo Llama.

Gerenciamento de Estado e Memória (Chat History): O código implementa a memória do agente através da lista *self.messages* dentro da classe *Agent*. Cada interação (usuário, pensamento da IA e observação das ferramentas) é anexada a essa lista, permitindo que o LLM mantenha o contexto de passos anteriores em problemas complexos.

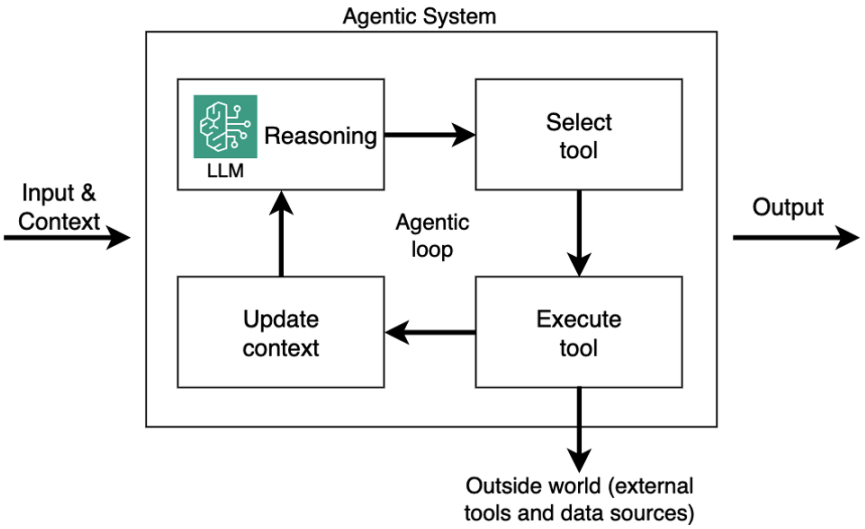
Implementação de Ferramentas (Tools): O vídeo explica que ferramentas são funções Python comuns mapeadas para o agente. Exemplos práticos incluem uma função de calculadora e uma função que retorna a massa de planetas (simulando uma busca em banco de dados ou API).

Processamento de Respostas com Regex: Como não se usa um framework, é necessário escrever código manualmente (utilizando expressões regulares) para identificar qual ação o modelo solicitou e extrair os argumentos necessários para executar a função correspondente no Python.

O Loop de Controle (Agent Loop): O apresentador codifica um laço *while* que automatiza todo o processo: envia a pergunta, captura a ação da IA, executa a ferramenta, devolve a "Observação" para o modelo e repete o ciclo até que a IA forneça uma "Resposta Final".



[Imagem 1]



[Imagem 2]

Em relação ao novo código criado, foi criado um novo *prompt* de sistema, que mantém a lógica de funcionamento ReAct, feito em português, o que altera a forma do modelo estruturar a resposta e foram definidas regras de negócio específicas, como "Sempre verifique o preço do voo antes de sugerir um orçamento". Foram criadas novas ações/ferramentas, funções que simulam serviços de viagem:

- **`buscar_voo(destino)`**: Em vez de uma busca genérica, esta função utiliza um dicionário mapeando cidades específicas (Roma, Tóquio, Lisboa, Nova York) para preços fixos em dólar (USD).
- **`converter_moeda(consulta)`**: Esta função utiliza **Expressões Regulares (Regex)** para extrair dinamicamente o valor e as moedas de uma string (ex: "100 USD para BRL"). Ela aplica taxas de conversão fixas para calcular o resultado em moeda local.

Diferente da versão da aula que usava o comando `eval()` de forma mais direta, utilizei um dicionário de ferramentas (tools) para gerenciar as ações: dentro do agent_loop_viagem, as funções são mapeadas em uma estrutura de chave-valor: {"buscar_voo": buscar_voo, "converter_moeda": converter_moeda}.

Dificuldades

Pensando nas maiores dificuldades encontradas, destaco o uso de uma linguagem da qual eu ainda não me aprofundei, o Python, mas com seu grande uso e popularidade não é difícil aprender suas particularidades, há muito conteúdo disponível na internet. Além disso, também me deparei com um erro no git ao tentar subir o commit com a adição dos arquivos, que descobri ser por conta do uso de uma chave, que é considerada um “segredo”, então tive que criar arquivos .env e .gitignore para solucionar o problema.

Conclusões

Os temas estudados foram de grande importância para compreender o funcionamento dos agentes antes de se aprofundar em frameworks e bibliotecas prontos. Agora, compreendo o funcionamento do ReAct e como funciona o modo de agir dos agentes de IA. A arquitetura reafirma a importância de um loop de controle bem definido e de um prompt de sistema rigoroso para garantir que a IA interaja com ferramentas externas de maneira precisa e autônoma.

Referencias

[Imagem 1] [Imagem 2] Blogs da AWS. Criação eficaz de agentes de IA no AWS Serverless. Disponível em: https://aws.amazon.com/pt/blogs/aws-brasil/criacao-eficaz-de-agentes-de-ia-no-aws-serverless/ . Acesso em: 05 mar. 2026

Alejandro AO. Python: Create a ReAct Agent from Scratch. Youtube, 2024. Disponível em: https://www.youtube.com/watch?v=hKVhRA9kfeM. Acesso em: 04 mar. 2026.